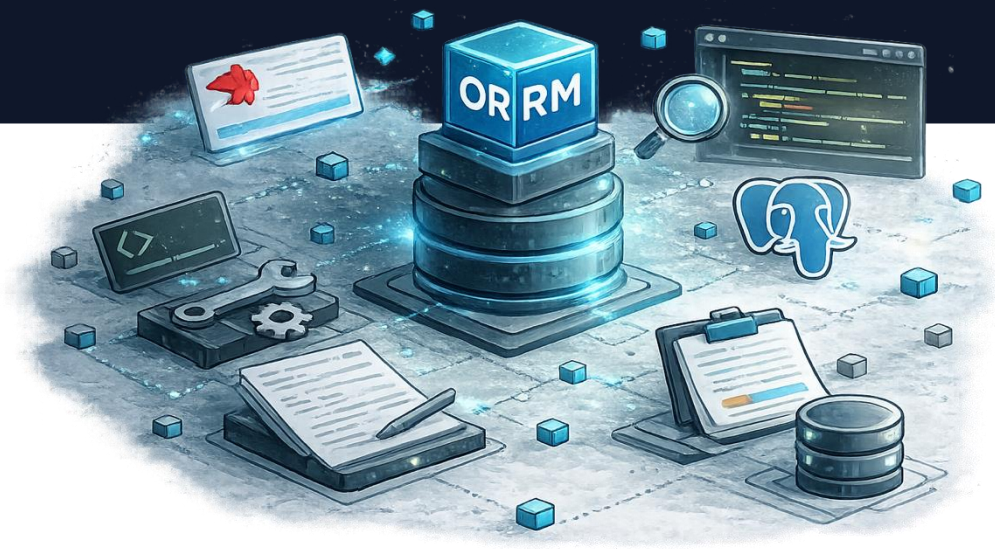
Progettazione di applicazioni web full stack

Laboratorio su TypeORM e PostgreSQL



Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia

Argomento di questa lezione…

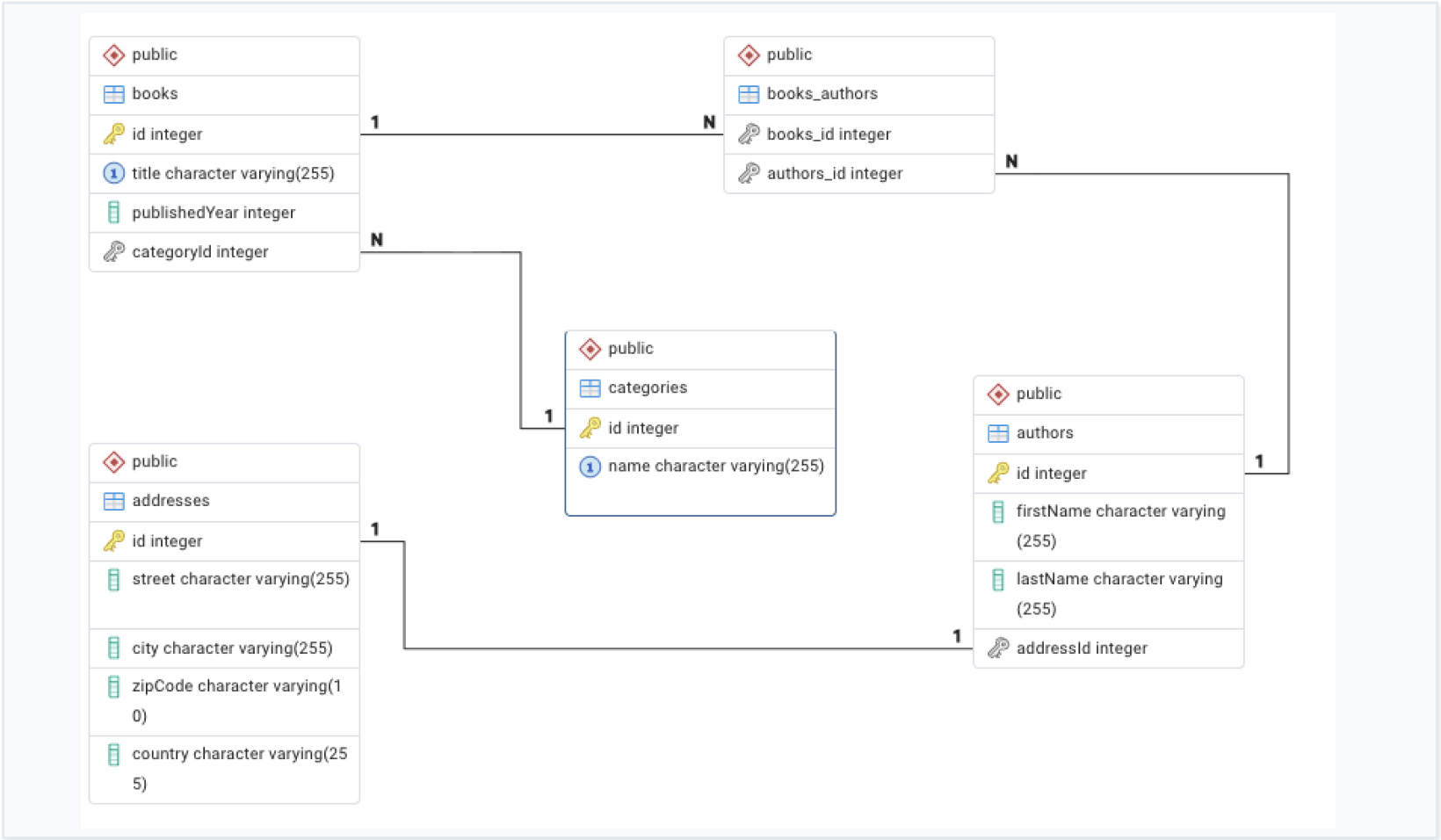
Cosa faremo

- Approfondimento dell'uso di **TypeORM**
- Utilizzo delle relazioni *uno-a-uno*, *uno-a-molti*, *molti-a-molti* tramite i *decorator* **TypeORM**
- Ottimizzazione del codice

Output

- Una libreria backend in **NestJS** e **TypeORM** per la persistenza di dati su una biblioteca (libri con categorie, autori e indirizzi)
- Estensione di un workspace esistente (`type-lab3`) tramite integrazione della nuova libreria di backend

DB Entity-Relation



Entità

- File **`address.entity.ts`**

Una **Entity** per rappresentare gli indirizzi (**`Address`**)

TypeORM *decorator* per identificare una *relazione uno-a-uno* su **`Author`**

```ts
import { PrimaryGeneratedColumn, Column, OneToOne, Entity } from 'typeorm';
import { Author } from "./author.entity";

@Entity('addresses')
export class Address {
    @PrimaryGeneratedColumn()
    id: number;

    @Column({type:'varchar', length: 255})
    street: string;

    @Column({type:'varchar', length: 255})
    city: string;

    @Column({type:'varchar', length: 10})
    zipCode: string;

    @Column({type:'varchar', length: 255})
    country: string;

    @OneToOne(()=>Author, (author)=>author.address)
    author: Author;
}
```

Entità

- File **`author.entity.ts`**

Una **Entity** per rappresentare gli indirizzi (**`Address`**)

Una **Entity** per rappresentare gli autori (**`Author`**)

TypeORM *decorator* per identificare una *relazione molti-a-molti* su **`Book`**

```
@Entity('authors')
export class Author {
    @PrimaryGeneratedColumn()
    id: number;

    @Column({type:'varchar', length: 255, nullable: false})
    firstName: string;

    @Column({type:'varchar', length: 255, nullable: false})
    lastName: string;

    @ManyToMany(()=>Book,(book)=>book.authors)
    books: Book[];

    @OneToOne(()=>Address, (address)=>address.author,{cascade:true, eager: true})
    @JoinColumn()
    address: Address;
}
```

Entità

- File **`author.entity.ts`**

Una **Entity** per rappresentare gli indirizzi (**`Address`**)

Una **Entity** per rappresentare gli autori (**`Author`**)

TypeORM *decorator* per identificare una *relazione uno-a-uno* su **`Address`** e posizionare sulla proprietà address il vincolo di chiave esterna

```
@Entity('authors')
export class Author {
    @PrimaryGeneratedColumn()
    id: number;

    @Column({type:'varchar', length: 255, nullable: false})
    firstName: string;

    @Column({type:'varchar', length: 255, nullable: false})
    lastName: string;

    @ManyToMany(()=>Book,(book)=>book.authors)
    books: Book[];

    @OneToOne(()=>Address, (address)=>address.author,{cascade:true, eager: true})
    @JoinColumn()
    address: Address;
}
```

Entità

- File **`author.entity.ts`**

Una **Entity** per rappresentare gli indirizzi (**`Address`**)

Una **Entity** per rappresentare gli autori (**`Author`**)

`cascade: true` indica che è possibile operare un'operazione di salvataggio, modifica, cancellazione su Author e Address in una volta sola (verrà mostrato un esempio tra poco)

Possibile specificare meglio con un array (e.g., **`['insert', 'update']`**, **`['remove']`**)

```
@Entity('authors')
export class Author {
    @PrimaryGeneratedColumn()
    id: number;

    @Column({type:'varchar', length: 255, nullable: false})
    firstName: string;

    @Column({type:'varchar', length: 255, nullable: false})
    lastName: string;

    @ManyToMany(()=>Book,(book)=>book.authors)
    books: Book[];

    @OneToOne(()=>Address, (address)=>address.author,{cascade:true, eager: true})
    @JoinColumn()
    address: Address;
}
```

Entità

- File **`author.entity.ts`**

Una **Entity** per rappresentare gli indirizzi (**`Address`**)

Una **Entity** per rappresentare gli autori (**`Author`**)

`eager: true` indica che quando leggo l'entità **`Author`**, viene caricata automaticamente anche la relazione, diversamente bisogna esplicitare (dentro il metodo **`find`** del **repository**)

```
relations: {
  address: true
}
```

```
@Entity('authors')
export class Author {
    @PrimaryGeneratedColumn()
    id: number;

    @Column({type:'varchar', length: 255, nullable: false})
    firstName: string;

    @Column({type:'varchar', length: 255, nullable: false})
    lastName: string;

    @ManyToMany(()=>Book,(book)=>book.authors)
    books: Book[];

    @OneToOne(()=>Address, (address)=>address.author,{cascade:true, eager: true})
    @JoinColumn()
    address: Address;
}
```

Entità

- File **`book.entity.ts`**


Una **Entity** per rappresentare gli indirizzi (**`Address`**)


Una **Entity** per rappresentare gli autori (**`Author`**)


Una **Entity** per rappresentare i libri (**`Book`**)

```typescript
@Entity('books')
export class Book {
    @PrimaryGeneratedColumn()
    id: number;

    @Column({type: 'varchar', length: 255, nullable: false, unique: true})
    title: string;

    @Column({type: 'int', nullable: false})
    publishedYear: number;

    @ManyToMany(()=>Author,(author)=>author.books, {cascade: false})
    @JoinTable({
        name: 'book_authors',
        joinColumn: {name: 'book_id', referencedColumnName:'id'},
        inverseJoinColumn: {name: 'author_id', referencedColumnName:'id'}
    })
    authors: Author[];

    @ManyToOne(()=>Category,(category)=>category.books,{nullable:false, eager:true,onDelete:'RESTRICT'})
    @JoinColumn() // optional on OneToMany relations
    category: Category;
}
```

Entità

- File **`book.entity.ts`**

Una **Entity** per rappresentare gli indirizzi (**`Address`**)

Una **Entity** per rappresentare gli autori (**`Author`**)

Una **Entity** per rappresentare i libri (**`Book`**)

```
@Entity('books')
export class Book {
    @PrimaryGeneratedColumn()
    id: number;

    @Column({type: 'varchar', length: 255, nullable: false, unique: true})
    title: string;

    @Column({type: 'int', nullable: false})
    publishedYear: number;

    @ManyToMany(()=>Author,(author)=>author.books, {cascade: false})
    @JoinTable({
        name: 'book_authors',
        joinColumn: {name: 'book_id', referencedColumnName:'id'},
        inverseJoinColumn: {name: 'author_id', referencedColumnName:'id'}
    Author[];
    )=>Category,(category)=>category.books,{nullable:false, eager:true,onDelete:'RESTRICT'})
    // optional on OneToMany relations
    ry;
```

TypeORM *decorator* per identificare una *relazione molti-a-molti* su **`Author`** e descrivere la tabella pivot che rappresenterà la relazione (non serve un'entità ulteriore)

Entità

- File **`book.entity.ts`**

Una **Entity** per rappresentare gli indirizzi (**`Address`**)

Una **Entity** per rappresentare gli autori (**`Author`**)

Una **Entity** per rappresentare i libri (**`Book`**)

```
@Entity('books')
export class Book {
    @PrimaryGeneratedColumn()
    id: number;

    @Column({type: 'varchar', length: 255, nullable: false, unique: true})
    title: string;

    @Column({type: 'int', nullable: false})
    publishedYear: number;

    @ManyToMany(()=>Author,(author)=>author.books, {cascade: false})
    @JoinTable({
        name: 'book_authors',
        joinColumn: {name: 'book_id', referencedColumnName:'id'},
        inverseJoinColumn: {name: 'author_id', referencedColumnName:'id'}
    })
    authors: Author[];

    @ManyToOne(()=>Category,(category)=>category.books,{nullable:false, eager:true,onDelete:'RESTRICT'})
    @JoinColumn() // optional on OneToMany relations
    category: Category;
}
```

- La foreign key non può essere **`null`** (**`nullable`** è **`false`**)
- Quando viene caricato un book, viene automaticamente caricata anche la category (**`eager: true`**)
- Se si tenta di cancellare un libro, viene cancellata anche la categoria *solo se non ci sono altri libri associati* (**`onDelete: 'RESTRICT'`**)

Entità

- File **`category.entity.ts`**


Una **Entity** per rappresentare gli indirizzi (**`Address`**)


Una **Entity** per rappresentare gli autori (**`Author`**)


Una **Entity** per rappresentare i libri (**`Book`**)


Una **Entity** per rappresentare le categorie (**`Category`**)

```typescript
import { Entity, PrimaryGeneratedColumn, Column, OneToMany } from 'typeorm';
import { Book } from './book.entity';

@Entity('categories')
export class Category {
    @PrimaryGeneratedColumn()
    id: number;

    @Column({type:'varchar', length: 255, nullable: false, unique: true})
    name: string;

    @OneToMany(()=>Book,(book)=>book.category)
    books: Book[];
}
```

Servizio e repository

- Il repository ha l'unico scopo di implementare le azioni CRUD sul DB
- Ottimizziamo riducendo il codice ridondante

- File **`books.service.ts`**

```
@Injectable()
export class OrgBooksService {
    constructor(
        @InjectRepository(Book)
        private readonly bookRepository: Repository<Book>,

        @InjectRepository(Author)
        private readonly authorRepository: Repository<Author>,

        @InjectRepository(Category)
        private readonly categoryRepository: Repository<Category>
    ) {}
```

Servizio e repository

- Metodo per il popolamento del database (file **`books.service.ts`**)

```
async seed() {
    try{
        const category = this.categoryRepository.create({
            name: 'Fantascienza'
        });
        const savedCategory = await this.categoryRepository.save(category);

        const author1 = this.authorRepository.create({
            firstName: 'Isaac',
            lastName: 'Asimov',
            address: {
                street: 'Via Roma 10',
                city: 'Milan',
                zipCode: '20100',
                country: 'Italy'
            }
        });

        const author2 = this.authorRepository.create({
            firstName: 'Arthur',
            lastName: 'Clarke',
            address: {
                street: 'Corso Torino 22',
                city: 'Turin',
                zipCode: '10100',
                country: 'Italy'
            }
        });

        const savedAuthors = await this.authorRepository.save([author1,author2]);
```

Servizio e repository

- Metodo per il popolamento del database (file **`books.service.ts`**)

```
            const book = this.bookRepository.create({
                title: 'Antology of the Sci-Fi',
                publishedYear: 2025,
                category: savedCategory,
                authors: savedAuthors
            });

            return await this.bookRepository.save(book);
        } catch(error) {
            this.handleDatabaseError(error);
        }
    }
```

Gestione «elegante», «trasparente» ed «estendibile» degli errori (si veda il codice **`typeorm-lab4`**)

Servizio e repository

- Metodo per la lettura delle informazioni sui libri nel database (file **`books.service.ts`**)

```
async findAllBooks() {
    return await this.bookRepository.find({
        relations: {
            authors: true
        }
    });
}
```

Necessario perché non è stato impostato **`eager: true`** nella relazione molti-a-molti tra **`Author`** e **`Book`**

Controller

- Esposizione delle funzionalità di caricamento e ispezione del database (file **`books.controller.ts`**)

```ts
import { Controller, Get, Post } from '@nestjs/common';
import { OrgBooksService } from './books.service';
import { ApiTags } from '@nestjs/swagger';

@ApiTags('Library APIs')
@Controller('books')
export class OrgBooksController {
  constructor(private orgBooksService: OrgBooksService) {}

  @Post('populate')
  populateDB() {
    return this.orgBooksService.seed();
  }

  @Get()
  findAll() {
    return this.orgBooksService.findAllBooks();
  }
}
```

Esportare la libreria…

- File **`books.module.ts`**

```ts
import { Address } from './address.entity';
import { Author } from './author.entity';
import { Book } from './book.entity';
import { Category } from './category.entity';

@Module({
  imports: [TypeOrmModule.forFeature([Address,Author,Book,Category])],
  controllers: [OrgBooksController],
  providers: [OrgBooksService],
  exports: [OrgBooksService],
})
export class OrgBooksModule {}
```

Il **repository** è *iniettato direttamente nel servizio*, non costituisce classe a parte

…per importarla in una qualsiasi app

- Nella app di backend

```
import { Module } from '@nestjs/common';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { ServerUsersModule } from '@server/users';
import { OrgBooksModule } from '@org/books';
import { DatabaseModule } from '@org/database';

@Module({
  imports: [ServerUsersModule, OrgBooksModule, DatabaseModule],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

Link e riferimenti utili

- Link alla documentazione TypeORM: https://typeorm.io/docs
- Integrazione di TypeORM in NestJS: https://docs.nestjs.com/techniques/database

Progettazione di applicazioni web full stack

Laboratorio su TypeORM e PostgreSQL

Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia